

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import classification_report, confusion_matrix
from imblearn.over_sampling import SMOTE
```

Import Dataset

```
df_train = pd.read_excel('https://docs.google.com/uc?export=download&id=1juY...')
df_test = pd.read_excel('https://docs.google.com/uc?export=download&id=1mTOg...')
```

```
df_train.head()
```

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	STATUS NIKAH	IPS 1	IPS 2	IPS 3	IPS 4
0	ANIK WIDAYANTI	PEREMPUAN	BEKERJA	28	BELUM MENIKAH	2.76	2.80	3.20	3.20
1	DWI HESTYNA PRIHASTANTY	PEREMPUAN	MAHASISWA	32	BELUM MENIKAH	3.00	3.30	3.14	3.14
2	MURYAARIEF	PEREMPUAN	BEKERJA	30	BELUM	2.50	2.30	2.70	2.70

Next steps:

[Generate code with df_train](#)
[New interactive sheet](#)

```
df_train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 379 entries, 0 to 378
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   NAMA                   379 non-null    object
1   JENIS KELAMIN          379 non-null    object
2   STATUS MAHASISWA        379 non-null    object
3   UMUR                   379 non-null    int64
4   STATUS NIKAH            379 non-null    object
5   IPS 1                   379 non-null    float64
6   IPS 2                   379 non-null    float64
7   IPS 3                   379 non-null    float64
8   IPS 4                   379 non-null    float64
9   IPS 5                   379 non-null    float64
10  IPS 6                   379 non-null    float64
11  IPS 7                   379 non-null    float64
12  IPS 8                   372 non-null    float64
```

```

13 IPK 376 non-null float64
14 STATUS KELULUSAN 379 non-null object
dtypes: float64(9), int64(1), object(5)
memory usage: 44.5+ KB

```

```
df_test.head()
```

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	STATUS NIKAH	IPS 1	IPS 2	IPS 3	IPS 4
0	UNAMA	LAKI - LAKI	MAHASISWA	24	BELUM MENIKAH	3.17	2.70	3.23	2.41
1	LEYLEA TRİYANA PRATIWI	PEREMPUAN	MAHASISWA	26	BELUM MENIKAH	3.60	3.50	3.42	2.85
	VERIS				BELUM				

Next steps:

[Generate code with df_test](#)[New interactive sheet](#)

```
df_test.info()
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 145 entries, 0 to 144
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   NAMA                   145 non-null    object
1   JENIS KELAMIN          145 non-null    object
2   STATUS MAHASISWA       145 non-null    object
3   UMUR                   145 non-null    int64
4   STATUS NIKAH           145 non-null    object
5   IPS 1                  145 non-null    float64
6   IPS 2                  145 non-null    float64
7   IPS 3                  145 non-null    float64
8   IPS 4                  145 non-null    float64
9   IPS 5                  145 non-null    float64
10  IPS 6                  145 non-null    float64
11  IPS 7                  145 non-null    float64
12  IPS 8                  141 non-null    float64
13  IPK                    142 non-null    float64
14  STATUS KELULUSAN       145 non-null    object
dtypes: float64(9), int64(1), object(5)
memory usage: 17.1+ KB

```

Data Cleaning

```

print(df_train['JENIS KELAMIN'].unique())
print(df_train['STATUS MAHASISWA'].unique())
print(df_train['STATUS NIKAH'].unique())
print(df_train['STATUS KELULUSAN'].unique())

```

```

['PEREMPUAN' 'LAKI - LAKI']
['BEKERJA' 'MAHASISWA']

```

```
['BELUM MENIKAH' 'MENIKAH']
['TERLAMBAT' 'TEPAT']
```

MENGHAPUS KOLOM YANG TIDAK DIGUNAKAN

```
df_train.drop(columns=['STATUS NIKAH'], inplace=True)
df_test.drop(columns=['STATUS NIKAH'], inplace=True)
```

```
df_train = df_train.dropna(subset=['IPS 8'])
df_train = df_train.dropna(subset=['IPK '])
```

```
df_test = df_test.dropna(subset=['IPS 8'])
df_test = df_test.dropna(subset=['IPK '])
```

```
df_train = df_train.drop(columns=['NAMA'])
df_train.head()
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8
0	PEREMPUAN	BEKERJA	28	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0
1	PEREMPUAN	MAHASISWA	32	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0
2	PEREMPUAN	BEKERJA	29	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0
3	PEREMPUAN	MAHASISWA	27	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0
4	PEREMPUAN	BEKERJA	28	3.00	3.00	3.00	3.05	3.00	3.00	3.00	0.0

Next steps:

[Generate code with df_train](#)
[New interactive sheet](#)

```
replacements = {
    'JENIS KELAMIN' : {'LAKI - LAKI' : 1, 'PEREMPUAN' : 0},
    'STATUS MAHASISWA' : {'MAHASISWA' : 0, 'BEKERJA' : 1},
    'STATUS KELULUSAN' : {'TERLAMBAT' : 1, 'TEPAT' : 0}
}
df_train = df_train.replace(replacements, inplace=False)
df_train.head()
```

```
/tmp/ipython-input-43836479.py:6: FutureWarning: Downcasting behavior in `rep
df_train = df_train.replace(replacements, inplace=False)
```

	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7	IPS 8	IPK
0	0	1	28	2.76	2.80	3.20	3.17	2.98	3.00	3.03	0.0	3.07
1	0	0	32	3.00	3.30	3.14	3.14	2.84	3.13	3.25	0.0	3.17
2	0	1	29	3.50	3.30	3.70	3.29	3.53	3.72	3.73	0.0	3.54
3	0	0	27	3.17	3.41	3.61	3.36	3.48	3.63	3.46	0.0	3.41
4	0	1	29	2.90	2.89	3.30	2.85	2.98	3.00	3.08	0.0	3.09

Next steps:

[Generate code with df_train](#)[New interactive sheet](#)

```
df_test = df_test.replace(replacements, inplace=False)
df_test.head()
```

```
/tmp/ipython-input-1812349956.py:1: FutureWarning: Downcasting behavior in `r
df_test = df_test.replace(replacements, inplace=False)
```

	NAMA	JENIS KELAMIN	STATUS MAHASISWA	UMUR	IPS 1	IPS 2	IPS 3	IPS 4	IPS 5	IPS 6	IPS 7
0	UNAMA	1	0	24	3.17	2.70	3.23	2.41	3.00	2.47	1.75
2	VERIS SOFIYAN PRAYOGA	1	0	29	2.67	2.66	2.93	3.14	2.92	2.64	2.88
3	ADITYA AKBAR NUGRAHA	1	0	27	2.48	2.86	2.09	2.55	2.55	2.43	2.55

Next steps:

[Generate code with df_test](#)[New interactive sheet](#)

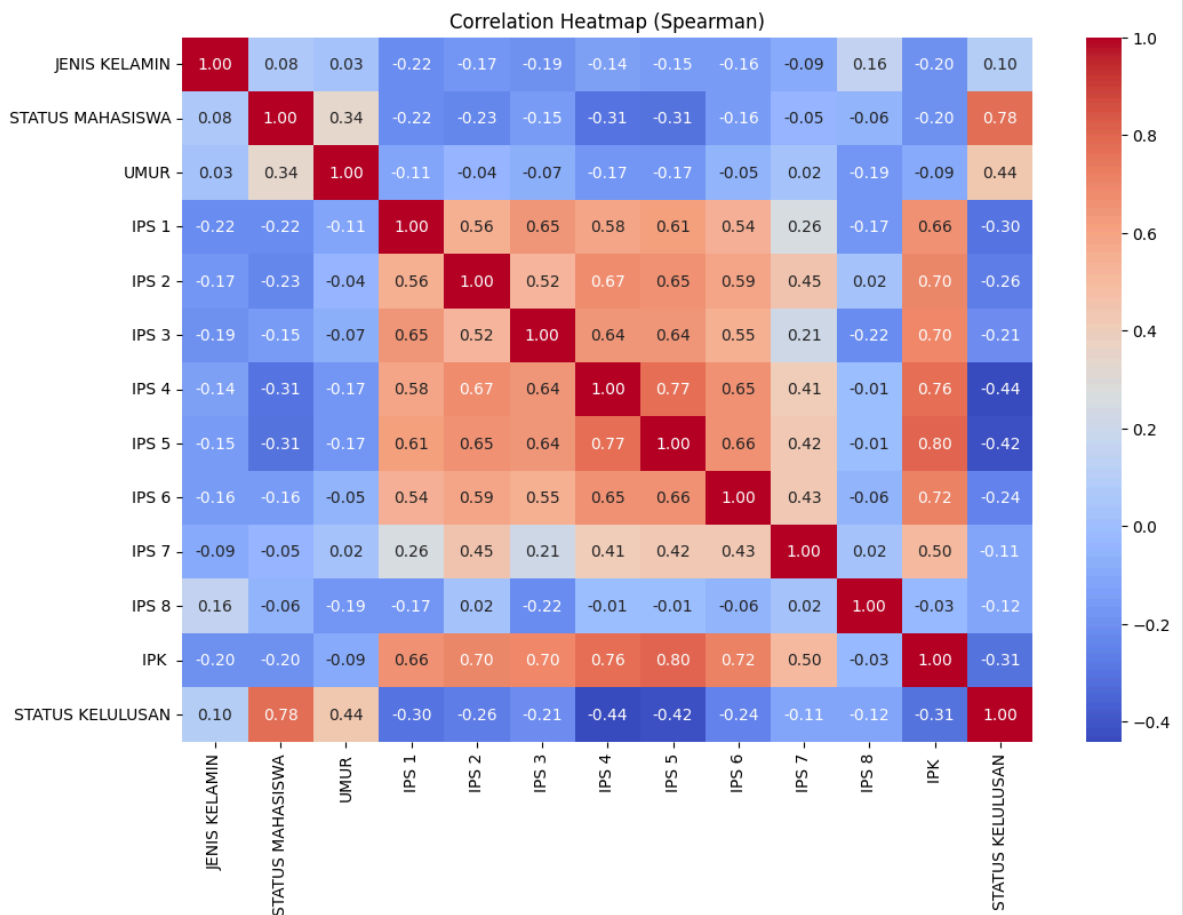
```
df_train.isnull().sum()
```

	0
JENIS KELAMIN	0
STATUS MAHASISWA	0
UMUR	0
IPS 1	0
IPS 2	0
IPS 3	0
IPS 4	0
IPS 5	0
IPS 6	0
IPS 7	0
IPS 8	0
IPK	0
STATUS KELULUSAN	0

dtype: int64

```
correlation_matrix = df_train.corr(method='spearman')

plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap='coolwarm', cbar=True)
plt.title('Correlation Heatmap (Spearman)')
plt.show()
```



```
x_train = df_train.drop(columns=['STATUS KELULUSAN'])
y_train = df_train['STATUS KELULUSAN']
```

```
x_test = df_test.drop(columns=['STATUS KELULUSAN'])
y_test = df_test['STATUS KELULUSAN']
```

```
scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
```

```
x_test = x_test.drop(columns=['NAMA'])
x_test_scaled = scaler.transform(x_test)
```

```
print("before SMOTE:")
print(y_train.value_counts())

sm = SMOTE(random_state=42, k_neighbors=1)
X_train_smote, y_train_smote = sm.fit_resample(x_train_scaled, y_train)

print("After SMOTE:")
print(y_train_smote.value_counts())
```

```
before SMOTE:
STATUS KELULUSAN
0      206
1      163
Name: count, dtype: int64
After SMOTE:
STATUS KELULUSAN
1      206
0      206
Name: count, dtype: int64
```

HYPERPARAMETER TUNING

```
param_grid = {
    'n_neighbors' : list(range(1, 31)),
    'weights' : ['uniform', 'distance'],
    'metric' : ['euclidean', 'manhattan']
}

grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5, n_jobs=-1)
grid.fit(X_train_smote, y_train_smote)

print("Best Params :", grid.best_params_)
print("Best Score (CV) :", grid.best_score_)
```

```
Best Params : {'metric': 'manhattan', 'n_neighbors': 19, 'weights': 'distance'}
Best Score (CV) : 0.9029973552747576
```

Melatih model KNN Terbaik dan Melakukan Prediks

```
best_knn = grid.best_estimator_
best_knn.fit(X_train_smote, y_train_smote)
# Prediksi
y_pred = best_knn.predict(x_test_scaled)
```

Evaluasi Model dengan Classification Report

```
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

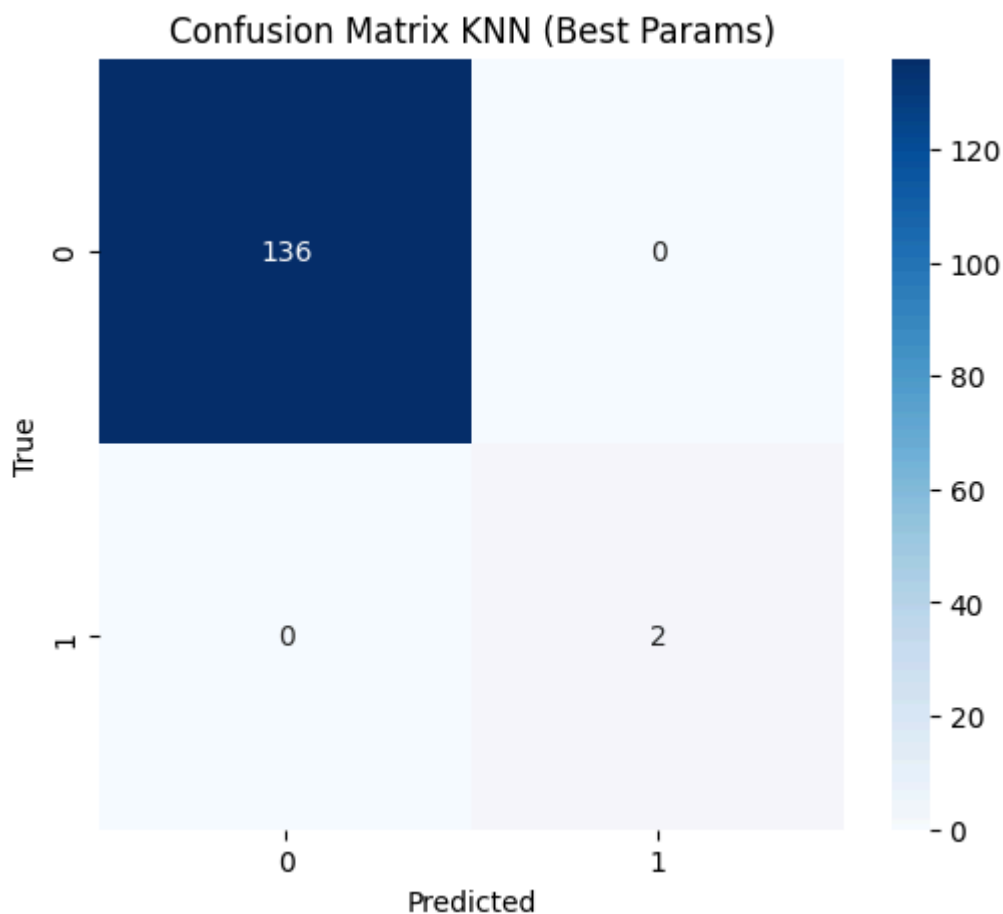
```
Classification Report:
              precision    recall  f1-score   support

     0           1.00        1.00        1.00        136
     1           1.00        1.00        1.00         2

   accuracy              1.00
  macro avg              1.00
 weighted avg              1.00
```

Evaluasi dengan Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(6,5))
sns.heatmap(cm, annot=True, cmap='Blues', fmt='d')
plt.title("Confusion Matrix KNN (Best Params)")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()
```



Validasi Model dengan Cross Validation


```
cv_scores = cross_val_score(best_knn, X_train_smote, y_train_smote, cv=5 )
print("Cross Validation Accuracy:", cv_scores.mean())
print("Std Dev:", cv_scores.std())
```

```
Cross Validation Accuracy: 0.9029973552747576
Std Dev: 0.042371318661662674
```

Menentukan Nilai Optimal dengan Elbow Method

```
errors = []

for k in range(1, 31):
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_smote, y_train_smote)
    pred = knn_temp.predict(x_test_scaled)
    errors.append(np.mean(pred != y_test))

plt.figure(figsize=(10,6))
plt.plot(range (1,31), errors, marker='o')
plt.title("Elbow Method for KNN")
plt.xlabel("K")
```